

Planificador por Lotería en Espacio de Usuario: Diseño, Verificación y Extensión con *Compensation Tickets*

Isaac Francisco Moreno Fuentes,
Emmanuel Barrantes Vargas

Resumen—Este informe documenta el diseño, la implementación y la verificación de un planificador de CPU por lotería (*lottery scheduling*) en espacio de usuario, inspirado en el trabajo de Waldspurger y Weihl [1]. El sistema coordina tareas concurrentes mediante *pthreadreads*, un mutex y variables de condición, sin espera activa ni un único bloqueo que impida demostrar el protocolo de despacho. Se presentan la arquitectura y el protocolo de sincronización, el pseudocódigo de los algoritmos centrales (sorteo, activación, cesión y terminación), la verificación contra los siete casos mínimos del enunciado bajo ThreadSanitizer, AddressSanitizer y LeakSanitizer, los resultados del experimento de proporcionalidad, la extensión elegida (*compensation tickets*) y su comparación con el paper original, y una discusión de los principales *trade-offs* de diseño.

Index Terms—lottery scheduling, proportional-share, pthreads, sincronización, compensation tickets

I. INTRODUCCIÓN

La planificación por lotería (*lottery scheduling*) representa derechos de uso del CPU mediante boletos: en cada decisión de despacho se sortea un boleto al azar entre las tareas activas y la propietaria del boleto ganador recibe el siguiente intervalo de ejecución. A largo plazo, si la selección es uniforme, la fracción de CPU que recibe una tarea converge a su fracción de los boletos activos — la propiedad de *proportional share* que motiva el mecanismo, descrita originalmente por Waldspurger y Weihl [1].

Este proyecto traduce esa política a un mecanismo ejecutable y verificable: un planificador en espacio de usuario, escrito en C17 sobre *pthread*s POSIX, que no modifica el *kernel* ni sustituye el planificador de GNU/Linux. El programa simula la existencia de un único CPU lógico — cada tarea es un hilo que solo avanza su trabajo cuando el planificador se lo permite explícitamente — y usa como carga de trabajo instrumentada el cálculo incremental de la serie de Taylor de $\arcsin(1) = \pi/2$, elegida por tener estado real que preservar entre activaciones (término anterior, suma acumulada, índice) y por ser verificable contra un valor conocido.

Isaac Francisco Moreno Fuentes y Emmanuel Barrantes Vargas son estudiantes del curso MC 6004 Sistemas Operativos Avanzados, Tecnológico de Costa Rica, II semestre 2026.

I-A. Alcance de este informe

Las secciones siguientes cubren, en orden: la arquitectura del sistema y el protocolo de sincronización (datos compartidos, mutex, variables de condición y sus predicados de espera); el pseudocódigo de los algoritmos centrales del sorteo, la activación, la cesión y la terminación; la verificación contra los siete casos mínimos de funcionamiento que exige el enunciado, incluyendo la evidencia de *sanitizers*; los resultados del experimento de proporcionalidad; el diseño e implementación de la extensión elegida (*compensation tickets*); un análisis comparativo con el paper original de Waldspurger y Weihl, incluyendo las diferencias con su prototipo sobre Mach 3.0 y las limitaciones de este prototipo en espacio de usuario; y una discusión de los *trade-offs* de diseño más relevantes. El código fuente completo, la trazabilidad del equipo y el historial de decisiones de diseño extendido viven en los repositorios públicos del proyecto, citados donde corresponde.

II. ARQUITECTURA Y PROTOCOLO DE SINCRONIZACIÓN

II-A. Modelo de hilos

El hilo principal (`main()`) actúa como el planificador; no existe un hilo dedicado separado para esta responsabilidad. El programa simula un único CPU lógico, por lo que no hay razón para que el planificador sea una entidad concurrente distinta del programa que lo invoca: esta decisión reduce en uno el número de hilos a crear, sincronizar y unir, y simplifica el argumento de terminación. Además del hilo principal, se crea un `pthread` por tarea (entre 5 y 25), cada uno ejecutando `worker_thread_main`. Todo hilo creado se une con `pthread_join` antes de que el programa termine.

II-B. Datos compartidos

Dos estructuras concentran todo el estado que más de un hilo puede tocar:

- **Task** (una por tarea): identificador, boletos base (`tickets`), boletos efectivos (`effective_tickets`, igual a `tickets` salvo compensación activa, Sección VI), estado (`READY/RUNNING/FINISHED`), trabajo total y ejecutado, contador de despachos, estado privado de la serie de π , y su propia variable de condición `cond_worker`.

- **Sync** (una para todo el programa): un `pthread_mutex_t`, la variable de condición `cond_scheduler` que espera el planificador, su predicado `event_ready`, y la bandera `stop_requested` que activa `-max-dispatches`.

Un único mutex (`sync.mutex`) protege el estado de todas las Task (`state`, `dispatch_count`, `effective_tickets`, `debt`), `event_ready` y `stop_requested`. Ese mutex **no** envuelve la ejecución de las unidades de trabajo de π : hacerlo violaría la prohibición explícita del enunciado de proteger toda la ejecución con un único bloqueo, y además sería innecesario. La Sección II-E explica por qué omitirlo ahí es seguro.

II-C. Variables de condición y predicados de espera

Se usa un esquema de **una cond_worker por tarea más una cond_scheduler compartida**, en vez de una única condvar global con un predicado de "turno" señalada por *broadcast*. La alternativa descartada despertaría innecesariamente a las $N - 1$ tareas que no ganaron cada despacho (*thundering herd*); el esquema elegido señala siempre de forma dirigida (`pthread_cond_signal`, nunca `pthread_cond_broadcast`) a un único hilo por evento.

Cuadro I: Condvars, predicados y quién señala

Condvar	Espera	Predicado	Señala
<code>task[i].cond_worker</code>	Worker i	<code>state==RUNNING</code> <code>stop_requested</code>	El planificador, al elegir a i como ganador (o <code>sync_request_stop</code> al cortar)
<code>cond_scheduler</code>	El planificador	<code>event_ready</code>	El worker en ejecución, al ceder o terminar

Ambos predicados se evalúan dentro de un `while`, nunca un `if`, por dos razones independientes: POSIX permite explícitamente que `pthread_cond_wait` retorne sin que nadie haya señalado la condición (un despertar sin causa aparente, conocido en la literatura como *spurious wakeup*); y el caso en que varios hilos comparten una condvar y el que despierta encuentra que la condición ya no aplica. `pthread_cond_wait` suelta el mutex y duerme el hilo de forma atómica, y vuelve a tomar el mutex antes de retornar — evita la condición de *señal perdida* que ocurriría si soltar el mutex y dormirse fueran dos pasos separados.

II-D. Protocolo de despacho

La Fig. 1 muestra el ciclo completo de un despacho. El planificador (hilo principal) ejecuta tres pasos en secuencia: `sync_select_winner` (sortea la ganadora entre las tareas `READY`, ponderando por `effective_tickets`), `sync_dispatch` (marca a la ganadora `RUNNING` y la señala), y `sync_wait_for_event` (se bloquea hasta que esa tarea ceda o termine). El worker ganador, mientras

tanto, estaba bloqueado en `sync_wait_for_dispatch`; al despertar, ejecuta su porción de trabajo **fuera del mutex** y cierra el ciclo con `sync_finish_turn`, que actualiza su estado y despierta al planificador.

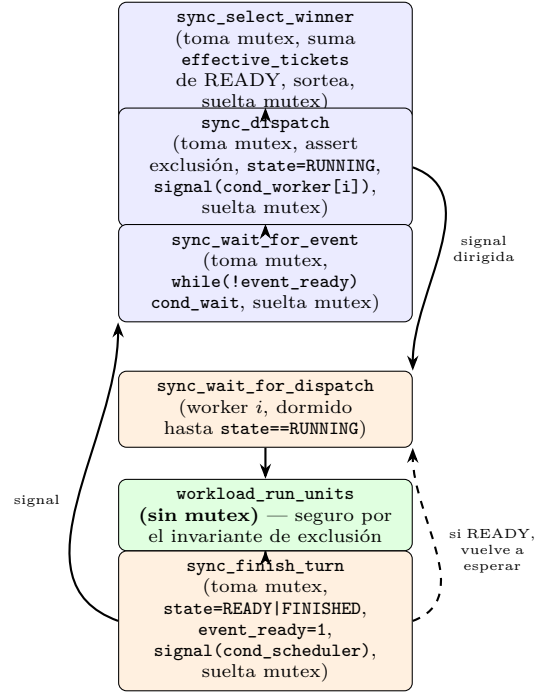


Figura 1: Un ciclo de despacho. Las cajas azules corren en el hilo planificador (**main**); las naranjas y la verde, en el hilo worker de la tarea ganadora. Solo el tramo verde corre sin el mutex tomado.

II-E. Por qué es seguro ejecutar el trabajo sin el mutex

El invariante de exclusión — como máximo una tarea en `RUNNING` en todo momento, garantizado por el `assert` dentro de `sync_dispatch` — implica que mientras una tarea corre, ningún otro hilo lee ni escribe sus campos: el planificador está bloqueado en `sync_wait_for_event` (no toca ninguna Task), y los demás workers están bloqueados en `sync_wait_for_dispatch` esperando su propia señal (nunca tocan una tarea ajena). El único hilo que puede tocar la tarea mientras corre es su propio dueño. Esta es una invariante del *protocolo* (el orden de llamadas), no algo que un *sanitizer* pueda verificar por sí solo — de ahí que cualquier código nuevo que necesite leer el progreso de una tarea `RUNNING` desde otro hilo (por ejemplo, un log en vivo) deba traer esa lectura dentro del mutex explícitamente, en vez de asumir que es seguro por analogía con este caso.

II-F. Terminación limpia y -max-dispatches

El enunciado exige que ningún trabajador quede bloqueado al finalizar. Una tarea que llega a `FINISHED` simplemente retorna de `worker_thread_main`. El caso menos obvio es `-max-dispatches`: cuando el planificador alcanza el límite, una tarea todavía `READY` que nunca vuelve a ser elegida se quedaría esperando su señal para

siempre. `sync_request_stop` resuelve esto señalando la `cond_worker` de cada tarea `READY` sin cambiar su estado; `sync_wait_for_dispatch` retorna un código distinto de cero en ese caso, y el worker retorna de inmediato sin ejecutar trabajo ni llamar `sync_finish_turn`, dejando su tarea intacta en `READY`. El planificador entonces une los N hilos con `worker_pool_join` sin que ninguno quede colgado.

III. ALGORITMOS

Esta sección presenta en pseudocódigo los cuatro algoritmos centrales del planificador: el sorteo ponderado, el bucle de despacho del planificador, el ciclo de un worker (que incluye la cesión y la actualización de boletos activos de la extensión), y el cálculo del tamaño de bloque cooperativo. El código real vive en `src/sync.c`, `src/scheduler.c` y `src/worker.c`.

Algorithm 1 Sorteo ponderado (`sync_select_winner`)

Require: tareas $T[1..N]$, generador rng

```

1: lock(mutex)
2: activos  $\leftarrow \sum_{i: T[i].state=READY} T[i].effective\_tickets$ 
3: if activos = 0 then
4:   unlock(mutex); return  $\perp$  {todas terminaron}
5: end if
6: boleto  $\leftarrow$  (rng.next() mód activos) + 1
7: acum  $\leftarrow$  0
8: for  $i \leftarrow 1$  to  $N$  do
9:   if  $T[i].state = READY$  then
10:    acum  $\leftarrow$  acum +  $T[i].effective\_tickets$ 
11:    if boleto  $\leq$  acum then
12:      unlock(mutex); return  $i$  {ganadora}
13:    end if
14:  end if
15: end for
```

El sesgo del módulo en la línea 6 (cuando activos no divide exacto el rango de `xorshift32`) se acepta y documenta en vez de corregirse con *rejection sampling*: para los tamaños de activos reales del proyecto (decenas a unos pocos miles) el sesgo relativo es del orden de 10^{-8} , siete órdenes de magnitud por debajo de la tolerancia del experimento de proporcionalidad ($\leq 0,02$).

El bloque cooperativo se calcula una sola vez sobre el `work_units total` de la tarea (no sobre el trabajo restante) y es constante durante toda su vida; en cada activación se topa por $\min(\text{bloque}, \text{restante})$, igual que el quantum fijo. Ambos modos comparten exactamente la misma función de ejecución (`workload_run_units`) y el mismo bucle del worker — la única diferencia mecánica entre cooperativo y quantum es cómo se calcula el tamaño del bloque, lo que explica por qué el Caso 6 (Sección IV) puede exigir resultados idénticos entre ambos modos con solo distinto costo de coordinación. La división se implementa con aritmética entera ($\lceil a/b \rceil = \lfloor (a + b - 1)/b \rfloor$) en vez de `double`, para preservar el determinismo bit a bit exigido por el enunciado.

Algorithm 2 Bucle del planificador (`scheduler_run`, hilo principal)

```

1: despachos  $\leftarrow$  0
2: loop
3:   if max_despachos > 0 and despachos  $\geq$  max_despachos then
4:     sync_request_stop() {despierta a las READY restantes sin despacharlas}
5:     break
6:   end if
7:    $g \leftarrow$  sync_select_winner()
8:   if  $g = \perp$  then
9:     break {ninguna tarea READY: todas terminaron}
10:  end if
11:  sync_dispatch( $g$ ) {state[ $g$ ]=RUNNING, signal dirigida a  $g$ }
12:  sync_wait_for_event() {bloquea hasta que  $g$  ceda o termine}
13:  despachos  $\leftarrow$  despachos + 1
14:  registrar evento de despacho (log)
15: end loop
16: worker_pool_join()
```

Algorithm 3 Ciclo del worker de la tarea t (`worker_thread_main`)

```

1: loop
2:   sync_wait_for_dispatch( $t$ ) {bloquea hasta RUNNING o stop}
3:   if el planificador solicitó parada antes de despachar a  $t$  then
4:     return { $t$  queda READY, sin ejecutar nada}
5:   end if
6:   bloque  $\leftarrow$  tamaño de bloque del modo (Algoritmo 4 en modo cooperativo, o quantum  $Q$  fijo)
7:    $u \leftarrow$  ACTUALIZARBOLETOS( $t$ , bloque) {Sección VI; sin -yield-config,  $u = \min(\text{bloque}, \text{restante})$ }
8:   workload_run_units( $t$ ,  $u$ ) {fuera del mutex}
9:   if  $t.completado = t.trabajo\_total$  then
10:    siguiente  $\leftarrow$  FINISHED
11:  else
12:    siguiente  $\leftarrow$  READY
13:  end if
14:  sync_finish_turn( $t$ , siguiente) {signal a cond_scheduler}
15:  if siguiente = FINISHED then
16:    return
17:  end if
18: end loop
```

Algorithm 4 Tamaño de bloque cooperativo (`cooperative_slice_size`)

Require: trabajo_total ≥ 1 , $1 \leq P \leq 100$

```

1: return  $\max\left(1, \left\lceil \frac{\text{trabajo\_total} \times P}{100} \right\rceil\right)$ 
```

IV. PRUEBAS Y VERIFICACIÓN

IV-A. Los siete casos mínimos de funcionamiento

La Tabla II resume los siete casos mínimos que exige la sección "Pruebas Mínimas de Funcionamiento" del enunciado, cada uno invocable de forma aislada desde `scripts/casos_enunciado/` y en conjunto vía `make casos-enunciado`. Los comandos exactos, los *fixtures* usados y la implementación de cada caso están documentados en el README del repositorio de código y en `milestone10_notas_tecnicas.md` del repositorio de conocimiento.

Cuadro II: Los 7 casos mínimos: comando, esperado, obtenido, conclusión

Caso	Esperado	Obtenido
1. Validación	Rechazo con código $\neq 0$ y sin ejecución parcial	8/8 verificaciones cumplen (los 4 escenarios del enunciado + evidencia de sin ejecución parcial)
2. Reproducibilidad	Dos corridas idénticas; las 5 tareas terminan	Logs idénticos byte a byte; 5/5 terminan
3. Igualdad	Share medio $\approx 0,20$, con desviación y error	Error absoluto medio 0,000477 (tolerancia 0,02), 30 semillas
4. Proporcionalidad	Shares objetivo 1/15...5/15; error $\leq 0,02$	Error absoluto medio 0,001104; shares 1/15...5/15 confirmados
5. Terminación	Ninguna FINISHED vuelve a ganar; suma correcta; joins completos	15/15 verificaciones cumplen
6. Modos	Mismo trabajo final y π entre modos	7/7 verificaciones cumplen; π idéntico bit a bit
7. Estrés	Sin deadlock, fuga, acceso inválido, <i>data race</i> ni UB	12/12 en las tres variantes de <i>sanitizer</i> (ver §IV-B)

Los Casos 1, 5 y 6 reportan únicamente el escenario que nombra el enunciado — no toda la suite de ingeniería heredada de los milestones donde se originaron (Milestone 1/8 para la validación de CLI/CSV, Milestone 5 para el bucle del planificador, Milestone 6 para los modos de ejecución). Esa cobertura adicional (52 verificaciones en total: 20 de validación extra de CLI/CSV, 20 del bucle del planificador, 39 de los modos de ejecución más 9 de la validación de `-yield-config`) se mantiene como parte del ciclo normal de desarrollo (`make test`), separada de la evidencia mínima que exige el enunciado.

IV-B. Evidencia de sanitizers

El Caso 7 (Estrés: 25 tareas, parámetros variados, 5 semillas en ambos modos más los casos borde de $Q = 1$ y `-max-dispatches`) se verificó en tres variantes, cada una en el entorno donde certifica algo distinto:

- **AddressSanitizer + UndefinedBehaviorSanitizer**, dentro de Docker (Ubuntu 24.04 x86_64, el

entorno de referencia del enunciado): 12/12, sin hallazgos.

- **AddressSanitizer + LeakSanitizer**, dentro de Docker: 12/12, sin fugas de memoria.
- **ThreadSanitizer**, fuera de Docker, directo en el host: 12/12, sin *data races*.

Estas tres variantes no corren en el mismo entorno por dos limitaciones conocidas, no atribuibles al código del proyecto: **LeakSanitizer no reporta fugas de forma confiable fuera de un Linux real** (un `malloc` deliberado sin `free`, compilado con `-fsanitize=address` y corrido fuera de Docker, termina con `exit=0` sin reportar nada; el mismo binario dentro de Docker sí lo detecta); y **ThreadSanitizer falla dentro de Docker en hosts ARM64** (FATAL: ThreadSanitizer: unexpected memory mapping) por un problema conocido de su instrumentación bajo la emulación x86_64 de Rosetta/QEMU, no relacionado con el código. Ninguna de las dos es un falso positivo del análisis en sí — son limitaciones del entorno de ejecución del *sanitizer*, mitigadas corriendo cada uno donde efectivamente funciona.

V. RESULTADOS EXPERIMENTALES

V-A. Experimento de proporcionalidad (Casos 3 y 4)

Ambos casos corren con 5 tareas de 10^7 unidades cada una, ventana truncada a 10^4 despachos (`-max-dispatches`) y quantum $Q = 1000$, sobre 30 semillas no nulas (1,...,30), reproducible con `scripts/experimento_proporcionalidad.py` (Milestone 11, Issue #13). La ventana truncada es necesaria para que la ponderación por boletos sea observable: si se deja correr hasta que todas las tareas terminan, `observed_share` converge a $1/N$ para todas independientemente de sus boletos, porque cada tarea termina consumiendo exactamente su trabajo total sin importar cuántas veces ganó el sorteo en el camino.

El Caso 3 (Igualdad, boletos iguales) confirma que sin diferencias de boletos el share converge a $1/5 = 0,20$ para las cinco tareas, con un error absoluto medio de 0,000477 sobre las 30 semillas (Tabla III).

Cuadro III: Caso 3 (Igualdad): share observado por tarea, 30 semillas

id	objetivo	media	desv. std.	error abs.
1	0.200000	0.199303	0.003660	0.000697
2	0.200000	0.200730	0.004186	0.000730
3	0.200000	0.199740	0.003769	0.000260
4	0.200000	0.200463	0.003250	0.000463
5	0.200000	0.199763	0.003909	0.000237

El Caso 4 (Proporcionalidad, boletos 10/20/30/40/50) es el caso ilustrativo: sus objetivos difieren por tarea (1/15,...,5/15), así que revela si el sorteo realmente pondera por boletos y no solo distribuye de forma uniforme. La Fig. 2 compara el share objetivo con el promedio observado sobre las 30 semillas, con la desviación estándar entre semillas como barra de error; el error absoluto medio entre

tareas es 0,001104, muy por debajo de la tolerancia del enunciado ($\leq 0,02$), y las barras de error son minúsculas con la ventana completa de 10^4 despachos.

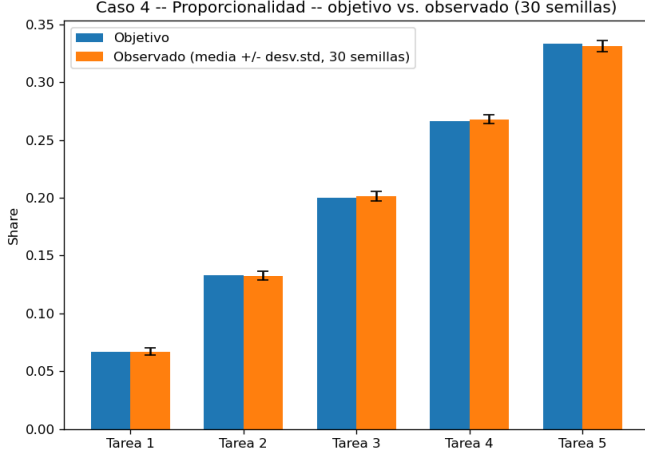


Figura 2: Caso 4 (Proporcionalidad): share objetivo vs. promedio observado por tarea, con desviación estándar entre las 30 semillas como barra de error. Generado por `scripts/experimento_proporcionalidad.py`.

V-B. Análisis 1: la lotería no garantiza exactitud en ventanas cortas

La Fig. 3 muestra el share acumulado de cada tarea después de cada despacho, de una semilla representativa (`seed=1`), desde el despacho 1 hasta el 10 000. En los primeros cientos de despachos las cinco líneas oscilan con amplitud grande y hasta se cruzan entre sí — la tarea 4 (40 boletos, objetivo 0,267) llega a tener share = 1,0 inmediatamente después del primer despacho, porque con una sola muestra quien gana tiene automáticamente el 100 % de lo acumulado hasta ese punto. A partir de aproximadamente el despacho 2000, cada línea se aplanan en una banda angosta alrededor de su objetivo (línea punteada) y prácticamente no se mueve más.

La razón estructural: `observed_share` de una tarea en el despacho k es un promedio acumulado (unidades corridas por esa tarea, dividido entre unidades corridas por todas, hasta k). Un promedio de pocas muestras está dominado por el resultado individual de esas muestras — el sorteo es *correcto en expectativa* desde el primer despacho (cada tirada respeta la ponderación de boletos, Sección VII), pero el *promedio observado* de pocas tiradas no tiene por qué parecerse todavía a esa expectativa: hace falta volumen para que el ruido de tiradas individuales se cancele entre sí.

V-C. Análisis 2: la variación cae como $1/\sqrt{N}$ con el número de despachos

Se corrió el mismo Caso 4 (30 semillas) variando `-max-dispatches`, para medir cómo cae la desviación estándar (promediada entre las 5 tareas) a medida que crece la ventana observada (Tabla IV).

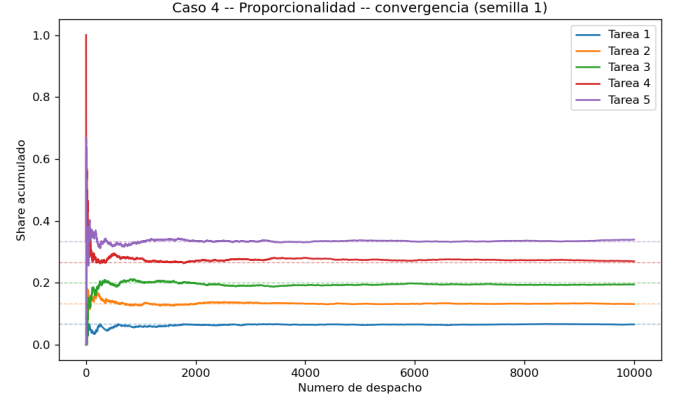


Figura 3: Caso 4: convergencia del share acumulado por tarea a lo largo de 10^4 despachos, semilla 1 (líneas punteadas: share objetivo de cada tarea). El Caso 3 exhibe el mismo patrón hacia un único objetivo común de 0,20 (`results/proporcionalidad/caso3_convergencia.png`).

Cuadro IV: Caso 4: error y desviación estándar según el tamaño de la ventana

Ventana (despachos)	Error absoluto medio	Desv. estándar media
50	0.006400	0.055580
100	0.004800	0.037241
500	0.001653	0.017224
1000	0.001680	0.011512
2500	0.001365	0.007220
5000	0.001205	0.005499
10000	0.001104	0.003978

La desviación estándar cae de 0,0556 a 0,0040 al pasar de 50 a 10 000 despachos (ventana 200 veces más grande) — una reducción de **13.97 veces**, prácticamente idéntica a $\sqrt{200} \approx 14,14$. Esto confirma que la variación entre semillas cae como $1/\sqrt{N}$ con el número de despachos N : cada despacho adicional es, a efectos de esta métrica, una observación más que se promedia con las anteriores, y el error estándar de un promedio de N observaciones aproximadamente independientes escala como $1/\sqrt{N}$. Es el mismo patrón de raíz cuadrada inversa documentado para el número de *semillas* en el experimento de la extensión (Sección V-E) — dos palancas relacionadas pero no intercambiables: más despachos reduce el ruido de la métrica dentro de una semilla; más semillas reduce la incertidumbre sobre la media de una métrica ya ruidosa por semilla.

V-D. Análisis 3: comparar shares entre tareas que terminan en momentos distintos puede engañar

`observed_share` se calcula sobre el total de unidades corridas por *todas* las tareas durante la ventana completa, lo que asume implícitamente que todas siguen compitiendo durante toda la ventana. Si una tarea termina su trabajo a mitad de camino y las demás siguen corriendo, el denominador sigue creciendo con el trabajo de las tareas restantes mientras el numerador de la que ya terminó queda fijo — su share final calculado sobre toda la ventana

queda artificialmente bajo, no porque el sorteo la haya tratado injustamente mientras competía, sino porque se le compara contra unidades corridas después de que dejó de competir. Es exactamente la razón por la que los Casos 3 y 4 usan tareas de 10^7 unidades cada una: con $\text{-max-dispatches} = 10^4$ y $\text{quantum} = 1000$, el trabajo total que cabe en la ventana está topado en $10^4 \times 1000 = 10^7$ unidades combinadas entre las 5 tareas — muy por debajo de lo que necesitaría cualquiera para agotar sus 10^7 unidades individuales, así que ninguna termina dentro de la ventana y todas compiten en igualdad de condiciones. El Caso 5 (Terminación) usa deliberadamente lo contrario — trabajos de tamaños distintos — pero por eso mismo no es un caso válido para medir proporcionalidad por share acumulado: sirve para verificar terminación correcta, no para comparar tasas de servicio entre tareas que dejan de competir en momentos distintos.

V-E. Experimento de la extensión (compensation tickets)

El diseño experimental (control $\text{-disable-compensation}$ vs. tratamiento con compensación activa) y las hipótesis H0/H1 se describen en la Sección VI. Se ejecutó `scripts/experiment_compensation_ab.py` con sus parámetros por defecto: tarea 3 de `tests/fixtures/valid_5.csv` (30 de 150 boletos, $\text{target_share} = 0.20$), modo *quantum* ($Q = 50$), $\text{yield_percent} = 25\%$, ventana de $\text{-max-dispatches} = 40$ sobre 100 semillas (1, ..., 100). La ventana se eligió deliberadamente por debajo de ~ 60 despachos — el número esperado de despachos para que la tarea con más boletos agote su trabajo asignado — de forma que ninguna tarea termine dentro de la ventana observada (misma razón que en el experimento de proporcionalidad, Sección anterior, pero aplicada al límite opuesto: aquí interesa una ventana corta para que la cesión temprana sea observable con pocos ciclos de cesión/compensación).

Cuadro V: Experimento A/B de compensation tickets: error absoluto de `observed_share` de la tarea 3 respecto a su target (0.20), control vs. tratamiento, 100 semillas

	media	desv. std.
error_A (sin compensar, control)	0.1395	0.0224
error_B (compensando, tratamiento)	0.0502	0.0343
mejora relativa $(\text{error}_A - \text{error}_B)/\text{error}_A$	+63.67 %	26.64 %

La mejora relativa promedio sobre las 100 semillas es de +63,67 %. Tratando las 100 semillas como muestras independientes, el error estándar de la media es $\text{SEM} = 0.2664/\sqrt{100} = 0.0266$, y el intervalo de confianza al 95 % es

$$0.6367 \pm 1.96 \times 0.0266 = [58.4\%, 68.9\%].$$

Como este intervalo no incluye 0 (ni valores cercanos), se rechaza H0: la compensación tiene un efecto medible y sustancial sobre `observed_share` de una tarea que cede tempranamente, consistente con H1.

La magnitud depende de la ventana, no solo su signo. Antes de fijar $\text{-max-dispatches} = 40$ como definitivo, se repitió el mismo experimento (100 semillas cada vez, para que la comparación entre ventanas sea justa) variando únicamente el tamaño de ventana:

Cuadro VI: Sensibilidad de la mejora relativa al tamaño de ventana (100 semillas por fila)

ventana (despachos)	mejora media	desv. std.
15	+3.3 %	2.15
20	+42.4 %	0.89
30	+55.4 %	0.35
40 (elegida)	+63.7 %	0.27
50	+69.4 %	0.28
55	+68.5 %	0.26

Dos hallazgos no anticipados al elegir la ventana de 40 únicamente por el criterio de "quedar bien por debajo del techo de ~ 60 despachos": con ventanas muy cortas (15 despachos) la métrica de mejora se vuelve matemáticamente inestable, porque su denominador (error_A , el error del control) puede caer cerca de cero por pura casualidad en semillas individuales, disparando el cociente a valores extremos (de ahí la desviación estándar de 2.15, mayor que la propia media); y la ventana original de un piloto preliminar de 20 despachos ya rechazaba H0 con una mejora positiva, pero por debajo del umbral de 50 % que se había fijado como criterio de éxito — el efecto es direccionalmente robusto en todo el rango probado (15-55 despachos, siempre positivo salvo el caso degenerado de 15), pero su magnitud reportada depende materialmente de cuántos ciclos completos de cesión/compensación alcanza a observar la ventana elegida. Un chequeo adicional variando yield_percent (15 %/25 %/35 %) y Q (30/50/70), siempre con 100 semillas, confirma que H0 se rechaza en las seis configuraciones probadas, con mejoras entre 52 % y 71 % — el efecto no depende de una combinación particular de parámetros, aunque tampoco converge a un único número.

VI. EXTENSIÓN: COMPENSATION TICKETS

VI-A. Extensión elegida y motivación

De las cinco extensiones propuestas por el enunciado (Tabla VII, Sección VII), el equipo eligió la **Opción A: compensation tickets**. La extensión exige "modelar una tarea que cede antes de consumir el *quantum* e inflar temporalmente su valor según la fracción consumida", y responde a la pregunta "¿recupera la participación de CPU a la que tenía derecho?". En el paper original [1], este mecanismo compensa a un servidor que se bloquea esperando E/S en nombre de un cliente y no debería ser penalizado frente a clientes que nunca se bloquean: al reactivarse, el servidor recibe temporalmente más boletos, proporcionales al inverso de la fracción de su último *quantum* que efectivamente usó, hasta agotar el equivalente a un *quantum* completo de CPU recibido en boletos compensados.

Se eligió esta opción sobre las otras cuatro porque reutiliza el mismo mecanismo cooperativo/quantum que

el proyecto base de todas formas debía implementar (no agrega una máquina de estados nueva ni una fuente de eventos externa), porque su pregunta de evaluación se responde con el mismo experimento de proporcionalidad que ya exige el proyecto (comparar `observed_share` con y sin compensación), y porque presenta menor superficie de bugs de concurrencia nuevos que la Opción C (estados cliente/servidor) o la D (doble contabilidad de monedas).

VI-B. El escenario debe forzarse

Con el diseño base del planificador (modos cooperativo y quantum tal como los especifica el enunciado), una tarea *siempre* consume todo su bloque asignado — no existe ningún mecanismo para que ceda antes de tiempo por cuenta propia. El escenario de cesión temprana que la extensión necesita observar no puede surgir orgánicamente; hay que modelarlo explícitamente, como pide el propio enunciado. Para eso se agregó un parámetro opcional y exclusivo del experimento de la extensión: `-yield-config <csv>`, un archivo con encabezado `task_id,yield_percent` que le indica a tareas puntuales que cedan tras correr solo una fracción de su bloque en vez de agotarlo. Ausente por defecto, no tiene ningún efecto sobre los siete casos mínimos del enunciado ni sobre el comportamiento base del planificador.

VI-C. Mecanismo

El *quantum* (o el bloque cooperativo) se mantiene fijo: lo que se infla temporalmente son los **boletos** de la tarea que cedió temprano, para darle mayor probabilidad de ganar los siguientes sorteos y así compensar el turno reducido. Si una tarea consumió solo una fracción f de su bloque antes de ceder:

$$\text{tickets_compensados} = \left\lceil \text{tickets_base} \times \frac{1}{f} \right\rceil \quad (1)$$

La compensación no es permanente: se mantiene mientras la tarea tenga una **deuda** pendiente, definida como lo que le faltó para completar el bloque original en la activación donde cedió. Cada vez que la tarea gana un sorteo con boletos inflados, corre el bloque completo (ya no vuelve a ceder mientras compensa) y descuenta lo corrido de la deuda; al llegar la deuda a cero, `effective_tickets` vuelve a `tickets_base` y el ciclo puede repetirse en la siguiente activación que gane.

VI-D. Decisiones de implementación

Tres puntos que el diseño original dejaba abiertos a propósito se resolvieron al implementar (detalle completo en `decisiones_diseno.md`, D15, del repositorio de conocimiento):

1. **La fracción f se mide contra el bloque que decide el modo, no contra un quantum fijo.** Con dos modos de ejecución, f se generaliza como `unidades_corridas/bloque_decidido`, donde `bloque_decidido` es la misma cantidad que ya calculaba `decide_slice_units` para esa activación

(quantum fijo, o el bloque cooperativo recortado por lo que falta) — sin ramas de código por modo dentro de la extensión.

2. **El campo `tickets` de Task no se renombra.** Ya cumple el rol de "boletos base"; se agregó `effective_tickets` como campo nuevo (igual a `tickets` salvo compensación activa) en vez de tocar los sitios que ya construían o leían `Task.tickets` desde el Milestone 1.
3. **La configuración de qué tareas ceden vive en un archivo aparte.** Nunca en el CSV principal de tareas: una bandera ausente por defecto es una garantía más fuerte de que los casos base no se ven afectados que una columna opcional que el parser tendría que aprender a ignorar.

El control del experimento A/B (aislar el efecto de *ceder* del efecto de *compensar*) tampoco estaba resuelto por el documento de diseño original: se agregó `-disable-compensation`, que requiere `-yield-config` y mantiene la cesión temprana activa pero nunca infla `effective_tickets` —la tarea cede la misma fracción en cada activación, indefinidamente, sin compensación— de modo que la comparación entre corridas con y sin esta bandera aísla exactamente el efecto de la compensación en sí.

VI-E. Hipótesis

- **H1.** Una tarea que cede voluntariamente antes de agotar su bloque, y que recibe *compensation tickets* proporcionales a la fracción no consumida, alcanza un `observed_share` significativamente más cercano a su share objetivo que la misma tarea bajo idénticas condiciones sin compensación.
- **H0.** La compensación no tiene efecto medible sobre el `observed_share` de una tarea que cede tempranamente.

El diseño experimental (control `-disable-compensation` vs. tratamiento con compensación activa, misma semilla, mismo CSV, misma ventana de despachos, repetido sobre múltiples semillas) y sus resultados se presentan en la Sección V-E.

VII. ANÁLISIS DEL PAPER

VII-A. Problema y solución

Waldspurger y Weihl [1] identifican que los mecanismos de planificación basados en prioridades (incluyendo esquemas de prioridad dinámica como el de Unix) no ofrecen un control *proporcional* y predecible sobre la asignación de recursos: ajustar prioridades es una operación ad-hoc, sin una relación cuantitativa clara entre el valor asignado y la fracción de recurso efectivamente recibida, y las políticas que sí intentan dar garantías proporcionales (como *fair-share scheduling* clásico) suelen requerir mecanismos de retroalimentación complejos o no componen bien cuando los derechos deben transferirse temporalmente entre entidades (por ejemplo, de un cliente bloqueado a un servidor que trabaja en su nombre).

La solución propuesta es la **planificación por lotería**: cada entidad recibe una cantidad de *boletos* que representa su derecho sobre el recurso, y cada decisión de asignación se resuelve sorteando un boleto al azar entre los boletos activos. El mecanismo es simple de implementar, componible (los boletos se pueden agrupar, transferir o subdividir sin lógica especial), y — a diferencia de los esquemas de prioridad — da una garantía *probabilística* de proporcionalidad en vez de una garantía determinista de corto plazo.

VII-B. Proporcionalidad probabilística

La proporcionalidad de la lotería es un resultado esperado, no una garantía por despacho: cada sorteo es un ensayo de Bernoulli independiente, y la fracción de sorteos ganados por una tarea con t_i boletos de un total activo T converge a t_i/T por la ley de los grandes números conforme aumenta el número de sorteos observados, con una varianza que decrece con ese mismo número. Es exactamente lo que exhibe la Fig. 3 (Sección V): en pocos cientos de despachos el share observado todavía oscila con amplitud visible; hacia el final de la ventana de 10^4 despachos converge de forma estable a la fracción objetivo. Este carácter probabilístico — y no una asignación exacta y determinista de cada intervalo — es la diferencia conceptual central entre la planificación por lotería y un planificador de cuotas fijas.

VII-C. Extensión elegida frente a las demás opciones

El enunciado ofrece cinco extensiones posibles sobre el mecanismo base (Tabla VII); el equipo eligió la Opción A (*compensation tickets*, Sección VI) por las razones ya expuestas: reutiliza el mecanismo cooperativo/quantum que el proyecto base debía implementar de todas formas, y su pregunta de evaluación se responde con el mismo experimento de proporcionalidad que ya exige el proyecto.

Las opciones descartadas se evaluaron y se prefirió A sobre ellas por menor superficie de bugs de concurrencia nuevos: C requiere modelar estados cliente/servidor adicionales a los tres estados de *Task* ya definidos; D requiere una doble contabilidad de monedas con su propia lógica de conversión; B modifica la asignación de boletos desde una fuente externa durante la corrida, lo que interactúa con la validación de entrada de formas que el enunciado no especifica. E se discute también en la Sección VIII como *trade-off* de diseño, ya que su pregunta de evaluación (¿en qué tamaño de carga compensa la complejidad $O(\log n)$?) es ortogonal a la elegida y aplica independientemente de qué extensión funcional se implemente.

VII-D. Diferencias con la implementación sobre Mach 3.0

El paper original valida *lottery scheduling* integrándolo al planificador de hilos del **microkernel Mach 3.0**, reemplazando su política de prioridades sobre hardware y cargas de trabajo reales: hilos del kernel, interrupciones de

Cuadro VII: Las 5 extensiones propuestas por el enunciado

Opción	Requisito mínimo
A. <i>Compensation tickets</i> (elegida)	Modelar una tarea que cede antes del quantum e inflar temporalmente su valor
B. Boletos dinámicos	Aplicar cambios de tickets en despachos definidos por un archivo de eventos
C. <i>Ticket transfer</i>	Modelar cliente bloqueado y servidor que recibe temporalmente sus boletos
D. <i>Currencies</i>	Dos monedas respaldadas por shares base e inflación local
E. Selección $O(\log n)$	Árbol de sumas parciales en vez de lista acumulativa

temporizador reales que disparan la expropiación, bloqueo real en E/S, y múltiples tipos de recurso (incluyendo memoria y ancho de banda de red) generalizados bajo el mismo mecanismo de boletos.

Este proyecto es, por diseño, una **simulación en espacio de usuario**: no se modifica el *kernel* ni se sustituye el planificador real de GNU/Linux, que sigue decidiendo los ciclos de CPU físicos por debajo sin que el programa lo sepa. Las diferencias concretas son:

- **Un único CPU lógico simulado**, aplicado mediante el protocolo de mutex/condvars de la Sección II, en vez de la expropiación real por interrupción de temporizador del kernel.
- **Carga de trabajo sintética e instrumentada** (la serie de Taylor de π) en vez de cargas de programas reales; se eligió precisamente porque su estado es verificable, no porque modele algo del mundo real.
- **Un único tipo de recurso** (unidades de CPU lógicas), sin la generalización a memoria o ancho de banda que el paper demuestra.
- **Sin cesión real por E/S**: la cesión temprana de la extensión se fuerza explícitamente vía `-yield-config` en vez de surgir de un bloqueo de E/S real, porque el prototipo no tiene E/S que modelar.

VII-E. Limitaciones del prototipo

Además de las diferencias estructurales con Mach 3.0, el prototipo tiene limitaciones propias de su alcance como proyecto de curso: opera sobre un conjunto fijo de tareas conocido de antemano en el CSV de entrada (el paper soporta creación dinámica de hilos con boletos heredados); el rango de 5 a 25 tareas es una cota de validación del enunciado, no una limitación inherente del mecanismo de lotería; la selección de la ganadora es $O(n)$ sobre la lista de tareas activas (Opción E, no implementada, discutida

como *trade-off* en la Sección VIII); y no se implementa *ticket transfer* (Opción C), por lo que el prototipo no demuestra directamente el mecanismo que el paper usa para resolver inversión de prioridad en llamadas cliente-servidor — solo su analogía más cercana, la compensación por cesión temprana.

VIII. DISCUSIÓN DE *TRADE-OFFS*

VIII-A. *Quantum fijo vs. bloque proporcional: costo de coordinación*

Los modos cooperativo y quantum son mecánicamente idénticos — comparten el mismo bucle del worker (Algoritmo 3) y la misma función de ejecución; solo difiere cómo se calcula el tamaño del bloque por activación (Sección III). Esa única diferencia tiene, sin embargo, un costo de coordinación medible y opuesto según el tamaño relativo de las tareas. Con quantum fijo Q , una tarea pequeña frente a Q puede terminar en una sola activación mientras una tarea grande necesita muchas; con un bloque proporcional al tamaño de cada tarea, todas necesitan aproximadamente el mismo número de activaciones sin importar su **work_units** total, a costa de que una tarea grande ceda el CPU con más frecuencia de la que cedería bajo un quantum fijo equivalente. El Caso 6 del enunciado lo hace explícito con una sola tarea de 60 unidades: al 10 % cooperativo (bloques de 6) necesita 10 activaciones; con quantum $Q = 25$ (bloques 25+25+10) necesita solo 3, para exactamente el mismo trabajo total y el mismo resultado de π . Más activaciones significan más ciclos de mutex/condvar y más filas de log por unidad de trabajo real completada — el *overhead* de coordinación del protocolo de la Sección II-D, que no aparece en el resultado final observable pero sí en el costo de producirlo.

VIII-B. *Reproducibilidad vs. realismo temporal*

El enunciado exige que la misma entrada, semilla, modo y parámetros produzcan exactamente el mismo registro de decisiones (determinismo). Esto se logra midiendo el trabajo en **work_units** lógicas — iteraciones discretas de la serie de π — en vez de tiempo real de CPU, y usando aritmética entera (Algoritmo 4) en vez de **double** para el cálculo del bloque cooperativo, evitando que la representación de punto flotante introduzca variación entre corridas idénticas por diferencias de redondeo entre plataformas. El costo de esa reproducibilidad es **realismo**: en un sistema real, el tiempo que una tarea recibe depende de interrupciones de temporizador, latencia de memoria, otros procesos del sistema operativo real corriendo por debajo, y variabilidad de *hardware* — ninguno de esos factores está presente aquí. La elección es deliberada y coherente con el objetivo del proyecto (verificar un protocolo de sincronización y una política de asignación, no medir rendimiento real de un sistema), pero implica que los números de *overhead* de coordinación de la sección anterior son comparables *entre sí* (mismo entorno lógico) y no directamente extrapolables al costo real de cambio de contexto de un *scheduler* de producción.

VIII-C. *Selección $O(n)$ por lista vs. árbol de sumas parciales $O(\log n)$*

El sorteo (Algoritmo 1) localiza a la ganadora recorriendo la lista de tareas **READY** y acumulando boletos hasta que el boleto sorteado cae en el rango de alguna — costo $O(n)$ por despacho, donde n es el número de tareas activas. La Opción E del enunciado (Tabla VII) propone sustituir esa lista por un árbol de sumas parciales, reduciendo la localización a $O(\log n)$. Para el rango de tareas que exige este proyecto (5 a 25), la diferencia entre $O(n)$ y $O(\log n)$ es de a lo sumo un factor de $25/\log_2 25 \approx 5$ comparaciones por despacho — indistinguible en la práctica frente al costo fijo de las dos llamadas a **pthread_mutex_lock/unlock** y la señal de condvar que rodean cada sorteo. La complejidad adicional de mantener un árbol balanceado (actualizar sumas parciales cada vez que una tarea cambia de estado o de boletos efectivos, en vez de solo recorrer un arreglo) solo se justificaría con cientos o miles de tareas activas simultáneas, un régimen muy distinto al de este proyecto; de ahí que se prefiriera reutilizar el mecanismo cooperativo/quantum (Opción A) en vez de optimizar una operación que no es el cuello de botella real a esta escala.

VIII-D. *Equidad de corto plazo vs. largo plazo*

La proporcionalidad de la lotería es, por diseño, una propiedad de largo plazo (Sección VII): si una corrida se deja avanzar hasta que todas las tareas terminan, **observed_share** converge a $1/N$ para todas, sin importar sus boletos, simplemente porque cada tarea termina consumiendo exactamente su trabajo total y las que tienen más boletos solo llegan ahí más rápido. La ponderación real por boletos únicamente es observable dentro de una **ventana truncada** (**-max-dispatches**) sobre trabajo suficientemente grande, como en los Casos 3 y 4 (Sección V). Esto implica un *trade-off* de medición, no solo de diseño: la elección del tamaño de la ventana observada determina qué se puede concluir sobre equidad — una ventana demasiado corta introduce más varianza de corto plazo (menos sorteos acumulados para que la ley de los grandes números converja); demasiado larga, y algunas tareas empiezan a terminar, sesgando el share de las que quedan. La elección de ventana usada en los Casos 3/4 de este informe (10^4 despachos sobre tareas de 10^7 unidades) se validó verificando que ninguna tarea llega a completar su trabajo dentro de la ventana (Sección V); la sensibilidad de este mismo *trade-off* es aún más pronunciada al medir el efecto de la extensión de compensación, cuyo análisis de robustez frente al tamaño de ventana está en elaboración (Sección V-E).

IX. CONCLUSIONES

El planificador implementado traduce la política de *proportional-share scheduling* de Waldspurger y Weihl a un mecanismo ejecutable y verificable en espacio de usuario, cumpliendo las restricciones de concurrencia del enunciado: exclusión mutua verificada en tiempo de ejecución (el **assert** de **sync_dispatch**), sin espera activa (todo

bloqueo pasa por variables de condición con predicado explícito), y sin un único candado que envuelva la ejecución del trabajo real. Los siete casos mínimos de funcionamiento cumplen, verificados bajo AddressSanitizer, UndefinedBehaviorSanitizer, LeakSanitizer y ThreadSanitizer sin hallazgos, y el experimento de proporcionalidad confirma que el sorteo pondera correctamente por boletos dentro de una ventana de observación truncada, con error absoluto medio muy por debajo de la tolerancia exigida.

La extensión elegida, *compensation tickets*, reutiliza el mecanismo cooperativo/quantum ya construido para modelar una tarea que cede tempranamente y recupera participación de CPU mediante boletos inflados temporalmente, sin alterar el comportamiento del planificador base cuando la extensión no está activa. La cuantificación estadística final de cuánta participación recupera — y su sensibilidad al tamaño de la ventana observada — se completa en la Sección V-E.

La comparación con el paper original deja claras las simplificaciones deliberadas de este prototipo: un único CPU lógico simulado en vez de integración real con un *kernel*, una carga de trabajo sintética e instrumentada en vez de programas reales, y un único tipo de recurso en vez de la generalización a memoria y red que demuestra Mach 3.0. Estas simplificaciones son coherentes con el objetivo del proyecto — verificar el protocolo de sincronización y la política de asignación, no medir rendimiento de un sistema real — y se documentan explícitamente en vez de presentarse como limitaciones no reconocidas.

DECLARACIÓN DE TRANSPARENCIA Y USO ÉTICO DE IA

Para la redacción de este informe se utilizó Claude Code (Anthropic) como herramienta auxiliar de apoyo: síntesis de las decisiones de diseño y hallazgos técnicos ya documentados por el equipo a lo largo del proyecto (repositorios *SOA-P1-GrupoIE* y *SOA-P1-Knowledge*), redacción del texto en español, generación del pseudocódigo y las figuras a partir del código fuente real y de los datos experimentales reproducidos para este documento, y revisión ortográfica y de formato. Todo el contenido técnico (arquitectura, algoritmos, resultados de pruebas y experimentos) se verificó contra el código fuente y las corridas reales del programa antes de incluirse.

REFERENCIAS

- [1] C. A. Waldspurger and W. E. Weihl, “Lottery scheduling: Flexible proportional-share resource management,” in *Proceedings of the 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI '94)*. Monterey, CA, USA: USENIX Association, 1994.